运动模糊点扩散函数的离散实现

蔡斌　数学 jd2401　1024001338

2026 年 11 月 5 日

1　连续模型与离散规则

匀速直线运动由轨迹长度和方向确定。本次作业实现函数 `motion_psf(LEN,THETA)`，并与 MATLAB R2023b 的 `fspecial('motion',LEN,THETA)` 对照。`LEN` 的单位为像素，`THETA` 从水平正方向逆时针起算，单位为度 [?]。

1.1　连续运动核

设轨迹总长为 $L > 0$，方向角为 θ，运动中点位于原点。曝光过程中各位置的贡献相同，连续 PSF 为

$$h_c(x, y) = \frac{1}{L} \int_{-L/2}^{L/2} \delta(x - t\cos\theta)\, \delta(y - t\sin\theta)\, \mathrm{d}t. \tag{1}$$

核的质量集中在运动线段上，且总积分为 1。离散计算还需确定像素中心、核的尺寸及端点权重。

1.2　MATLAB 的离散构造

本次以本机 R2023b 中 `fspecial` 的 `motion` 分支为离散规则参照 [?]。按该实现，先把长度截到至少 1，并采用如下半长与角度：

$$\ell = \max(1, \mathrm{LEN}), \qquad a = \frac{\ell - 1}{2}, \qquad \phi = \frac{\mathrm{mod}(\mathrm{THETA}, 180)}{180}\pi. \tag{2}$$

其中 a 表示离散线段的半端点间距；连续积分中的 $L/2$ 表示轨迹半长。整数长度的水平核包含 `LEN` 个像素中心，其首末中心的间距为 $\mathrm{LEN} - 1$。

记 $c = \cos\phi$、$s = \sin\phi$、$\sigma = \mathrm{sign}(c)$，$\varepsilon =$ `eps`。半核网格的边界为

$$b_x = \mathrm{fix}(ac + \sigma - \ell\varepsilon), \qquad b_y = \mathrm{fix}(as + 1 - \ell\varepsilon), \tag{3}$$

其中 `fix` 向零取整，横坐标取 $0{:}\sigma{:}b_x$，纵坐标取 $0{:}b_y$。由角度决定网格后，水平核与竖直核分别退化为行向量和列向量。

对网格点 (x, y)，先计算有符号垂距 $u = yc - xs$。当 $\sqrt{x^2 + y^2} \geq a$ 且 $|u| \leq 1$ 时，用 $\sqrt{u^2 + \{a - |(x + us)/c|\}^2}$ 替换 u，计入端点距离。再以 $w = \max(1 + \varepsilon - |u|, 0)$ 得到半核权重，镜像展开为 W，并计算

$$h = \frac{W}{\sum_{i,j} W_{ij} + \varepsilon\ell^2}. \tag{4}$$

最后按矩阵行方向调整上下顺序。边界取整、端点计算次序和浮点修正项均保留在第 3 节函数中。

2 AI 初版与修改分析

直接要求 AI 根据长度和角度编写 PSF 后，得到以下按像素方格内轨迹长度积分的实现。该函数保留为 `motion_psf_ai.m`，用于与最终函数对照。

```
function h = motion_psf_ai(len, theta)
% Motion PSF for a length-len segment at a counterclockwise Cartesian angle.
validateattributes(len, {'numeric'}, {'real','finite','scalar','positive'});
validateattributes(theta, {'numeric'}, {'real','finite','scalar'});
d = [cosd(theta), sind(theta)];
p0 = -len*d/2;
p1 =  len*d/2;
xmin = floor(min(p0(1),p1(1)) - 0.5);
xmax = ceil (max(p0(1),p1(1)) + 0.5);
ymin = floor(min(p0(2),p1(2)) - 0.5);
ymax = ceil (max(p0(2),p1(2)) + 0.5);
h = zeros(ymax-ymin+1, xmax-xmin+1);
for yy = ymin:ymax
    for xx = xmin:xmax
        q = [xx, yy]; lo = 0; hi = 1;
        for k = 1:2
            if d(k) == 0
                if abs(q(k)-p0(k)) > 0.5, lo = 1; hi = 0; end
            else
                t = sort((q(k)+[-0.5 0.5]-p0(k))/d(k));
                lo = max(lo,t(1)); hi = min(hi,t(2));
            end
        end
        h(ymax-yy+1,xx-xmin+1) = len*max(0,hi-lo);
    end
end
h = h/sum(h(:));
[r,c] = find(h > 0);
h = h(min(r):max(r),min(c):max(c));
end
```

初版以 $p_0 = -\mathrm{LEN}\, d/2$ 为起点，却在裁剪时采用 $p(t) = p_0 + td$、$0 \le t \le 1$。由于 d 是单位向量，实际裁剪的轨迹只有 1 像素长。保留端点定义，将第 20 行的分母由 `d(k)` 改为 `p1(k)-p0(k)`，即可让参数区间覆盖整条线段。

表 1: `LEN=9` 时的初版与参数化修正结果

角度	初版尺寸	修正后尺寸	内置尺寸	修正后最大误差
0°	1×1	1×9	1×9	2.637×10^{-16}
45°	2×2	7×7	7×7	5.743×10^{-2}
90°	1×1	9×1	9×1	2.637×10^{-16}

修正后尺寸一致，但三个例子的 `isequal` 均为假。原因是按像素方格内的轨迹长度积分，与按像素中心距离分配权重采用了不同的离散规则。其中 45° 的差异达到 0.05743，不能归因于浮点舍入。因此，最终函数采用第 1 节给出的半核、端点权重及归一化规则。

3 最终函数

根据第 1 节的离散规则，将方形网格改为角度相关的半核网格，并统一端点修正、镜像和归一化步骤。函数不调用 `fspecial`；该内置函数只在测试脚本中充当参照。默认参数为 LEN=9、THETA=0。

```
function h = motion_psf(len, theta)
%MOTION_PSF Motion-blur PSF compatible with MATLAB R2023b fspecial.
% Discretization follows the R2023b fspecial motion branch.
% LEN is in pixels; THETA is counter-clockwise from the horizontal axis.
narginchk(0, 2);
if nargin < 1, len = 9; end
if nargin < 2, theta = 0; end
validateattributes(len, {'double'}, ...
    {'positive','finite','real','nonempty','scalar'}, mfilename, 'LEN', 1);
validateattributes(theta, {'double'}, ...
    {'finite','real','nonempty','scalar'}, mfilename, 'THETA', 2);
len = max(1, len);
halfLength = (len - 1) / 2;
phi = mod(theta, 180) / 180 * pi;
cosPhi = cos(phi);
sinPhi = sin(phi);
xSign = sign(cosPhi);
lineWidth = 1;

% Smallest half-grid for this angle.
sx = fix(halfLength*cosPhi + lineWidth*xSign - len*eps);
sy = fix(halfLength*sinPhi + lineWidth - len*eps);
[x, y] = meshgrid(0:xSign:sx, 0:sy);

% Perpendicular distance, with endpoint correction.
distToLine = y*cosPhi - x*sinPhi;
radialDistance = sqrt(x.^2 + y.^2);
lastPixel = find((radialDistance >= halfLength) & ...
    (abs(distToLine) <= lineWidth));
parallelEndpointDistance = halfLength - abs( ...
    (x(lastPixel) + distToLine(lastPixel)*sinPhi) / cosPhi);
distToLine(lastPixel) = sqrt( ...
    distToLine(lastPixel).^2 + parallelEndpointDistance.^2);

% Convert distance to weights, mirror, and normalize.
distToLine = lineWidth + eps - abs(distToLine);
distToLine(distToLine < 0) = 0;
h = rot90(distToLine, 2);
h(end+(1:end)-1, end+(1:end)-1) = distToLine;
h = h ./ (sum(h(:)) + eps*len*len);
if cosPhi > 0, h = flipud(h); end
end
```

4 结果验证

在 MATLAB R2023b（23.2.0.2365128）中，先比较输出尺寸，再用 `isequal` 检查逐元素一致性；对同尺寸矩阵另记录 $\max_{i,j} |h_{ij} - h_{ij}^{\mathrm{ref}}|$。测试分为三组：

1. 常规组合：长度 $[0.25, 1, 1.25, 2.5, 5, 9, 13.7, 20]$，角度 $[-180, -90, -45, 0, 30, 45, 89.5, 90, 135, 180, 225,$ 共 104 组。
2. 边界组合：11 个长度与 35 个角度，共 385 组。长度覆盖 ε、1 与 9 的邻域及 100；角度覆盖 $-180°$ 至 $360°$ 间各 $90°$ 倍数及其左右邻域。
3. 随机参数：1000 组，长度在 $[0.05, 100.05)$、角度在 $[-720, 720)$ 内均匀抽取。随机数生成器为 `twister`，种子为 20261105。

另检查无参数调用和省略角度的调用。实测结果见表 **??**。

表 2: 最终函数与 `fspecial` 的对照结果

测试范围	逐元素一致	最大绝对误差
常规参数组合	104/104	0
边界参数组合	385/385	0
随机参数组合	1000/1000	0
默认参数调用	2/2	0

边界与随机测试的 1385 个输出均为有限非负系数，满足中心对称；系数和与 1 的最大偏差为 2.243×10^{-14}。归一化中保留的浮点修正项与内置实现一致。

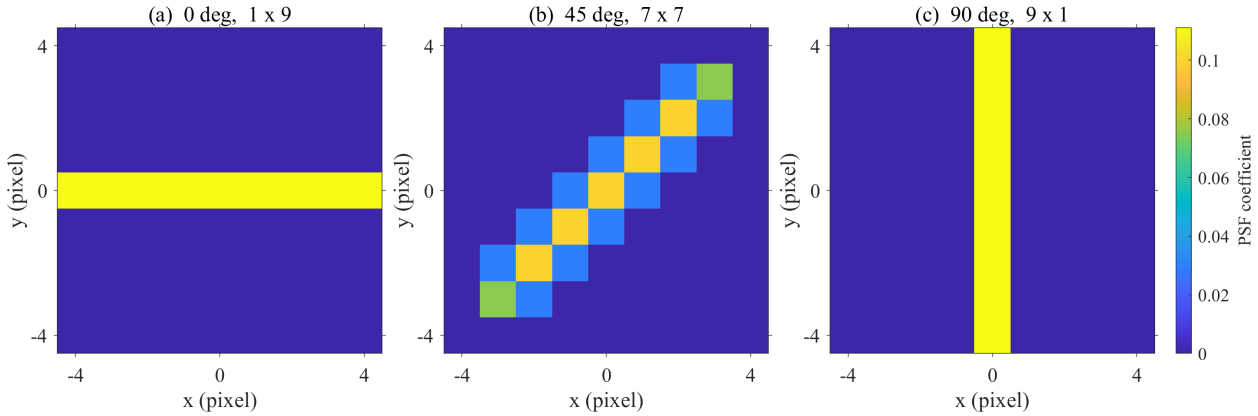


图 1: `LEN=9` 时的三个方向核。原始尺寸分别为 1×9、7×7、9×1；居中补零至 9×9 后按直角坐标系显示，共用色标。

图 **??** 中 $45°$ 的主轴从左下指向右上，与角度定义一致。本次 1489 组参数和 2 种默认调用均复现了 R2023b 的矩阵输出。实现的关键是将连续运动模型落实为一致的网格、端点权重和浮点计算顺序。

参考资料

[1] MathWorks. *fspecial: Create predefined 2-D filter*. `https://www.mathworks.com/help/images/ref/fspecial.html`.

[2] MathWorks. MATLAB R2023b，`fspecial.m` 的 `motion` 分支，第 190–225 行，本机安装文件。